RAPORT DE LABORATOR

Python: funcții, metode și importuri pe web

Importuri • metode • funcții • module • raport web

Student: Grosu Bogdan

Grupa: SI 262

Data: 11.11.2026

Universitate / facultate: Universitatea Tehnică a Moldovei - Facultatea Calculatoare, Informatică și Microelectronică (FCIM)

Prezentarea lucrării

În cadrul laboratorului au fost realizate exerciții Python care folosesc cereri HTTP către un site real. Lucrarea urmărește trei concepte principale - importurile, metodele și funcțiile - și continuă cu utilizarea modulelor din biblioteca standard, organizarea codului într-un modul propriu și generarea unui raport despre un site web.

Notă despre cod: Fragmentele de cod incluse în raport sunt selectate și simplificate pentru a evidenția partea relevantă a fiecărui exercițiu. Ele păstrează logica soluției; fișierele complete se găsesc în arhiva ZIP a proiectului.

Mediul de lucru și configurarea

Versiune Python: 3.13.3, conform capturii din terminal.

Biblioteca requests: versiunea 2.34.2; comanda pip install requests a indicat că pachetul și dependențele erau deja instalate.

Site-uri folosite: https://cybercor.org pentru cererile de citire și https://httpbin.org pentru verificarea antetului User-Agent personalizat.

Parametri comuni: BASE_URL = "https://cybercor.org", ECHO_URL = "https://httpbin.org" și TIMEOUT = 10 secunde.

Reguli respectate în soluție: cererile au timeout; între cererile repetate sunt introduse pauze time.sleep(1); pentru cybercor.org se folosesc cereri GET/HEAD; testul cu antet personalizat este trimis către httpbin.org.

Figura 1. Configurarea mediului: verificarea versiunii Python și a instalării bibliotecii requests.

```
PS D:\A SI\tp\web_lab> python --version
Python 3.13.3

PS D:\A SI\tp\web_lab> pip install requests
Requirement already satisfied: requests in c:\users\bogdan\appdata\local\programs\python\python313\lib\site-packages (2.34.2)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\bogdan\appdata\local\programs\python\python313\lib\site-packages (from requests) (3.5.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\bogdan\appdata\local\programs\python\python313\lib\site-packages (from requests) (3.20)
Requirement already satisfied: urllib3<3,>=1.26 in c:\users\bogdan\appdata\local\programs\python\python313\lib\site-packages (from requests) (2.8.0)
Requirement already satisfied: certifi>=2023.5.7 in c:\users\bogdan\appdata\local\programs\python\python313\lib\site-packages (from requests) (2026.10.23)
```

Structura raportului

- Partea 1 - Importuri (exercițiile 1-8), în fișierul part1_imports.py.
- Partea 2 - Metode și atribute ale obiectelor web (exercițiile 9-20), în part2_methods.py.
- Partea 3 - Scrierea propriilor funcții (exercițiile 21-34), în part3_functions.py.
- Partea 4 - Alte module din biblioteca standard (exercițiile 35-43), în part4_modules.py.
- Partea 5 - Construirea propriului modul și proiectul final (exercițiile 44-50), în webtools.py și main.py.
- Bonus - Rescrierea funcțiilor de extragere cu BeautifulSoup, în bonus_bs4.py.

Partea 1. Importuri (exercițiile 1-8)

Scop: Sunt prezentate modalitățile de importare a modulelor Python, diferența dintre biblioteca standard și pachetele externe și consultarea documentației.

Fișier: part1_imports.py

Exercițiul 1. Primul import

Cerință: Importarea bibliotecii requests și afișarea versiunii sale, apoi importarea urllib.request. În comentariu se explică de ce instalarea prin pip este necesară doar pentru requests.

Implementare: fragmentul relevant din soluția realizată:

```
import requests
print(requests.__version__)
import urllib.request
```

Explicație: requests este un pachet extern și se instalează separat, de exemplu cu pip install requests. urllib face parte din biblioteca standard Python și este disponibil automat după instalarea limbajului.

Rezultat observat: În captură este afișată versiunea requests 2.34.2.

Exercițiul 2. Două stiluri de import

Cerință: Descărcarea paginii principale și afișarea codului HTTP folosind atât requests.get(), cât și forma from requests import get.

Implementare: fragmentul relevant din soluția realizată:

```
import requests
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)

from requests import get
response = get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)
```

Explicație: Primul stil păstrează prefixul requests., ceea ce arată clar de unde provine funcția și reduce riscul de conflict între nume. Al doilea stil permite apeluri mai scurte, deoarece funcția get este importată direct în spațiul curent.

Rezultat observat: Ambele variante au returnat codul de stare 200.

Exercițiul 3. Alias-uri

Cerință: Importarea bibliotecii requests sub numele rq și folosirea rq.get() pentru descărcarea paginii.

Implementare: fragmentul relevant din soluția realizată:

```
import requests as rq
response = rq.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)
```

Explicație: Un alias scurtează numele unui modul și poate fi util când este o convenție cunoscută, precum np pentru NumPy. Un alias neobișnuit sau prea scurt, cum este rq, poate reduce lizibilitatea, deoarece cititorul trebuie să rețină la ce modul se referă.

Rezultat observat: Cererea realizată cu rq.get() a primit codul 200.

Exercițiul 4. Doar biblioteca standard

Cerință: Descărcarea paginii folosind urllib.request.urlopen(), afișarea atributului status și a primelor 200 de caractere din conținut.

Implementare: fragmentul relevant din soluția realizată:

```
request = urllib.request.Request(BASE_URL, headers={"User-Agent": "WebLab-Bogdan"})
with urllib.request.urlopen(request, timeout=TIMEOUT) as answer:
    print(answer.status)
    body = answer.read().decode("utf-8", errors="replace")
    print(body[:200])
```

Explicație: Metoda read() returnează date binare (bytes). decode("utf-8") transformă octeții într-un șir de caractere, astfel încât HTML-ul să poată fi afișat și prelucrat ca text. Blocul with închide automat răspunsul după utilizare, iar timeout limitează timpul de așteptare. Într-o încercare inițială fără antet User-Agent, serverul a răspuns cu 403; în varianta finală a fost adăugat User-Agent: WebLab-Bogdan, iar captura arată răspunsul 200.

Rezultat observat: Captura arată codul de stare 200 și începutul corpului HTML al paginii.

Exercițiul 5. Priviți în interiorul unui modul

Cerință: Afișarea numelor disponibile în requests cu dir() și clasificarea a trei elemente ca funcție, clasă sau modul.

Implementare: fragmentul relevant din soluția realizată:

```
print(dir(requests))
inspect.isfunction(requests.get)
inspect.isclass(requests.Session)
inspect.ismodule(requests.exceptions)
```

Explicație: dir(requests) listează numele expuse de modul. Pentru verificarea tipului au fost folosite funcțiile din modulul inspect: get este funcție, Session este clasă, iar exceptions este modul.

Rezultat observat: Verificările afișate în terminal au confirmat cele trei clasificări.

Exercițiul 6. Citiți documentația

Cerință: Rularea help(requests.get), identificarea parametrului care limitează timpul de așteptare și folosirea lui într-o cerere.

Implementare: fragmentul relevant din soluția realizată:

```
help(requests.get)
response = requests.get(BASE_URL, timeout=TIMEOUT)
```

Explicație: Parametrul timeout stabilește limita de timp pentru așteptarea răspunsului. El previne blocarea programului pe o cerere care nu se încheie. În implementare este folosită constanta TIMEOUT, cu valoarea de 10 secunde.

Rezultat observat: Cererea cu timeout=10 a returnat codul 200. Captura include și documentația funcției requests.get.

Exercițiul 7. Cronometrarea unei cereri

Cerință: Măsurarea duratei unei cereri cu time.perf_counter() și compararea ei cu response.elapsed.

Implementare: fragmentul relevant din soluția realizată:

```
start = time.perf_counter()
```

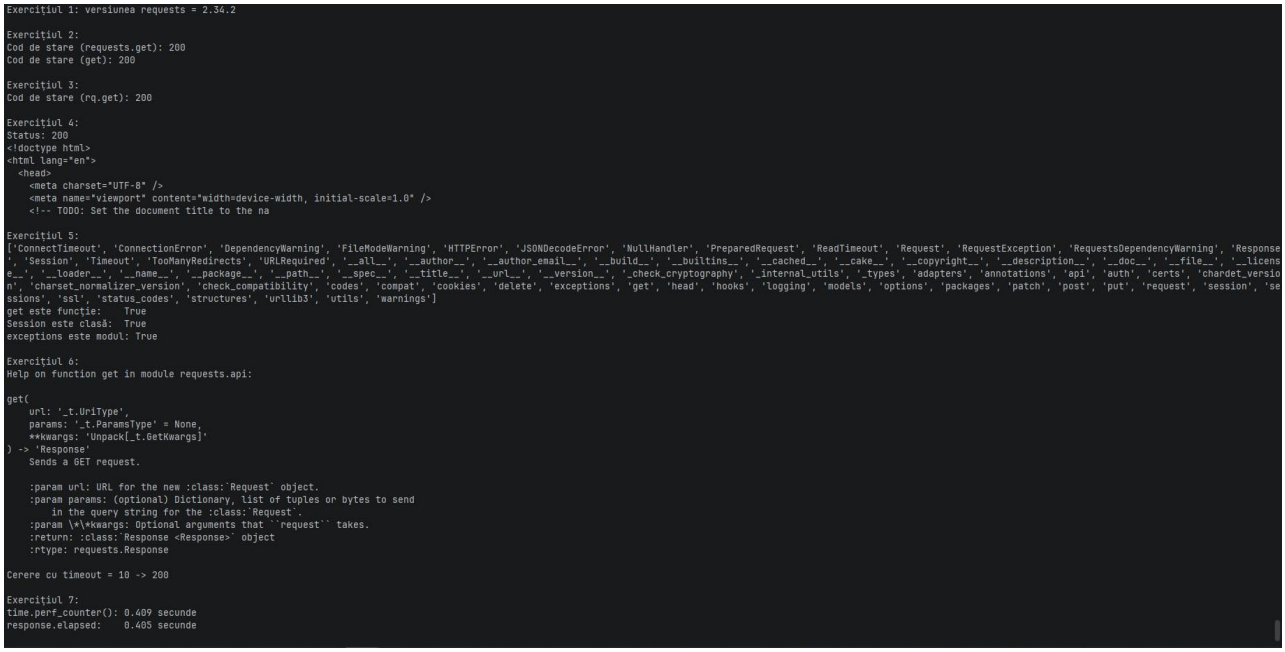
```
response = requests.get(BASE_URL, timeout=TIMEOUT)
end = time.perf_counter()

print(end - start)
print(response.elapsed.total_seconds())
```

Explicație: perf_counter() măsoară durata totală dintre începutul și sfârșitul apelului efectuat în program. response.elapsed reprezintă timpul raportat de requests pentru intervalul cererii până la sosirea antetelor răspunsului. Valorile pot fi apropiate, dar nu trebuie să fie identice, deoarece măsoară intervale diferite.

Rezultat observat: Ambele durate au fost afișate în secunde, cu trei zecimale.

Figura 2. Output comun pentru exercițiile 1-7 din part1_imports.py.



Exercițiul 8. Când un import eșuează

Cerință: Încercarea de importare a modulului bs4 într-un bloc try/except ImportError și afișarea unei instrucțiuni de instalare dacă modulul lipsește.

Implementare: fragmentul relevant din soluția realizată:

```
try:
    import bs4
    print("Modulul bs4 este instalat, versiunea", bs4.__version__)
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```

Explicație: Blocul try execută importul. Dacă Python nu găsește pachetul, excepția ImportError este prinsă și programul afișează comanda necesară. Astfel, lipsa unei biblioteci nu oprește programul cu un mesaj de eroare necontrolat.

Rezultat observat: Modulul bs4 era deja instalat, iar terminalul a afișat versiunea 4.15.0.

Figura 3. Output pentru exercițiul 8: BeautifulSoup este instalat.



Partea 2. Metode și atribute ale obiectelor web (exercițiile 9-20)

Scop: Se lucrează cu obiectul Response returnat de requests și cu atributele, metodele, antetele, cookie-urile și sesiunile sale.

Fișier: part2_methods.py

Exercițiul 9. Elementele de bază ale răspunsului

Cerință: Afișarea codului de stare, a indicatorului ok, a URL-ului final și a codificării răspunsului. Fiecare este identificat drept atribut.

Implementare: fragmentul relevant din soluția realizată:

```
response = requests.get(BASE_URL, timeout=TIMEOUT)
print(response.status_code)
print(response.ok)
print(response.url)
print(response.encoding)
```

Explicație: Toate cele patru elemente sunt atribute: reprezintă valori deja disponibile în obiectul Response și se citesc fără paranteze. status_code indică rezultatul HTTP, ok arată dacă răspunsul este considerat reușit, url este adresa finală, iar encoding indică setul de caractere folosit la decodarea textului.

Rezultat observat: Pagina a răspuns cu status_code 200, ok=True, URL HTTPS și encoding utf-8.

Exercițiul 10. Tratarea erorilor cu o metodă

Cerință: Apelarea metodei raise_for_status() pentru pagina principală și pentru o adresă care nu există, cu prinderea excepției requests.HTTPError.

Implementare: fragmentul relevant din soluția realizată:

```
response.raise_for_status()
missing = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
try:
    missing.raise_for_status()
except requests.HTTPError:
    print("Pagina nu există", missing.status_code)
```

Explicație: raise_for_status() nu face nimic pentru coduri de succes și ridică HTTPError pentru răspunsuri HTTP de eroare (4xx sau 5xx). Excepția este prinsă pentru ca programul să poată afișa un mesaj prietenos și să continue. În captura acestui laborator, ruta de test a returnat tot codul 200, deci ramura except nu a fost activată; aceasta este o particularitate a răspunsului serverului, nu o dovadă că metoda nu funcționează.

Rezultat observat: Pagina principală a răspuns cu 200, iar și ruta de test a fost raportată cu 200 în captura terminalului.

Exercițiul 11. Toate antetele

Cerință: Parcurgerea perechilor nume-valoare din response.headers.items() și afișarea fiecărui antet.

Implementare: fragmentul relevant din soluția realizată:

```
for name, value in response.headers.items():
    print(f"{name}: {value}")
```

Explicație: Antetele conțin informații suplimentare despre răspuns, de exemplu serverul care l-a trimis, tipul conținutului, politica de cache și unele antete de securitate. items() permite parcurgerea lor ca perechi cheie-valoare.

Rezultat observat: Au fost afișate antete precum Server, Content-Type, Cache-Control și X-Frame-Options.

Exercițiul 12. Metode de dicționar

Cerință: Folosirea metodei get() pentru antetele Server și Content-Type, cu valoarea implicită „lipsește”, apoi căutarea Content-Type cu litere mici.

Implementare: fragmentul relevant din soluția realizată:

```
response.headers.get("Server", "lipsește")
response.headers.get("Content-Type", "lipsește")
response.headers.get("content-type", "lipsește")
```

Explicație: get() returnează valoarea cheii dacă aceasta există sau valoarea implicită dacă nu există. response.headers este un CaseInsensitiveDict, astfel încât numele antetelor sunt căutate fără diferențiere între majuscule și minuscule.

Rezultat observat: Căutarea Content-Type și content-type a returnat aceeași valoare.

Exercițiul 13. Înlănțuirea metodelor de șir

Cerință: Numărarea aparițiilor cuvântului cyber în HTML folosind lower() și count().

Implementare: fragmentul relevant din soluția realizată:

```
count = response.text.lower().count("cyber")
print(count)
```

Explicație: lower() produce un șir nou cu toate literele transformate în minuscule. Pentru că rezultatul este tot un șir, se poate apela imediat count() pe el. count() numără aparițiile textului căutat fără a modifica șirul inițial.

Rezultat observat: Cuvântul cyber apare de 9 ori în textul analizat, conform capturii.

Exercițiul 14. Găsirea titlului

Cerință: Extragerea textului dintre tagurile <title> și </title> cu find(), slicing și strip().

Implementare: fragmentul relevant din soluția realizată:

```
tag_pos = html.find("<title>")
end = html.find("</title>")
if tag_pos != -1 and end != -1:
    start = tag_pos + len("<title>")
```

```
    title = html[start:end].strip()
    print(title)
```

Explicație: find() caută poziția de început a fiecărui tag. După găsirea pozițiilor, slicing extrage doar partea dintre taguri. strip() elimină spațiile și liniile goale de la marginea textului. Verificarea valorii -1 evită extragerea incorectă dacă lipsește unul dintre taguri.

Rezultat observat: Titlul extras este CYBERCOR.

Exercițiul 15. Numărarea liniilor

Cerință: Împărțirea HTML-ului în linii cu splitlines(), apoi calcularea numărului de linii și a lungimii celei mai lungi linii.

Implementare: fragmentul relevant din soluția realizată:

```
lines = html.splitlines()
print(len(lines))
if lines:
    print(len(max(lines, key=len)))
```

Explicație: splitlines() creează o listă în care fiecare element reprezintă o linie din HTML. max(lines, key=len) selectează linia cu lungimea maximă, iar len() măsoară numărul de caractere al acelei linii. Condiția if lines previne apelarea lui max() pe o listă goală.

Rezultat observat: În captură au fost afișate 32 de linii, iar cea mai lungă linie are 218 caractere.

Exercițiul 16. Verificarea HTTPS

Cerință: Verificarea dacă URL-ul final începe cu https:// folosind startswith().

Implementare: fragmentul relevant din soluția realizată:

```
if response.url.startswith("https://"):
    print("Conexiune securizată")
else:
    print("Conexiune nesecurizată")
```

Explicație: startswith() verifică prefixul unui șir și returnează True sau False. În acest caz se verifică dacă adresa folosită după eventualele redirecționări utilizează schema HTTPS.

Rezultat observat: Terminalul a afișat „Conexiune securizată”.

Figura 4. Output comun pentru exercițiile 9-16 din part2_methods.py.



Exercițiul 17. Urmărirea redirecționărilor

Cerință: Trimiterea unei cereri către varianta HTTP a paginii și afișarea fiecărei redirecționări, apoi a URL-ului final.

Implementare: fragmentul relevant din soluția realizată:

```
redirected = requests.get("http://cybercor.org", timeout=TIMEOUT)
for step in redirected.history:
    print(step.status_code, step.url)
print("URL final:", redirected.url)
```

Explicație: requests urmărește automat redirecționările. response.history conține răspunsurile intermediare, iar response.url conține adresa la care s-a ajuns în final.

Rezultat observat: A fost observată o redirecționare 301 de la http://cybercor.org/ către https://cybercor.org/.

Exercițiul 18. HEAD versus GET

Cerință: Compararea lungimii corpului răspunsului obținut cu requests.head() și requests.get().

Implementare: fragmentul relevant din soluția realizată:

```
head_response = requests.head(BASE_URL, timeout=TIMEOUT)
get_response = requests.get(BASE_URL, timeout=TIMEOUT)
print(len(head_response.content))
print(len(get_response.content))
```

Explicație: HEAD solicită doar antetele răspunsului, fără corpul paginii, de aceea content este gol. GET solicită atât antetele, cât și corpul HTML. HEAD este util când este nevoie de informații despre resursă fără descărcarea conținutului complet.

Rezultat observat: HEAD: 0 octeți; GET: 1540 octeți.

Exercițiul 19. Cookie-uri

Cerință: Parcurgerea cookie-urilor primite și afișarea atributelor name și secure. Dacă lista este goală, se afișează un mesaj corespunzător.

Implementare: fragmentul relevant din soluția realizată:

```
cookies_found = False
for cookie in response.cookies:
    cookies_found = True
    print(cookie.name, cookie.secure)
if not cookies_found:
    print("Niciun cookie setat")
```

Explicație: Fiecare cookie este un obiect cu propriile atribute. name indică numele cookie-ului, iar secure arată dacă transmiterea lui este limitată la conexiuni securizate. Variabila cookies_found permite detectarea unei liste goale.

Rezultat observat: Răspunsul paginii nu a conținut cookie-uri; terminalul a afișat „Niciun cookie setat”.

Exercițiul 20. Sesiuni

Cerință: Crearea unei sesiuni requests, setarea unui User-Agent personalizat și verificarea antetului primit de httpbin.org.

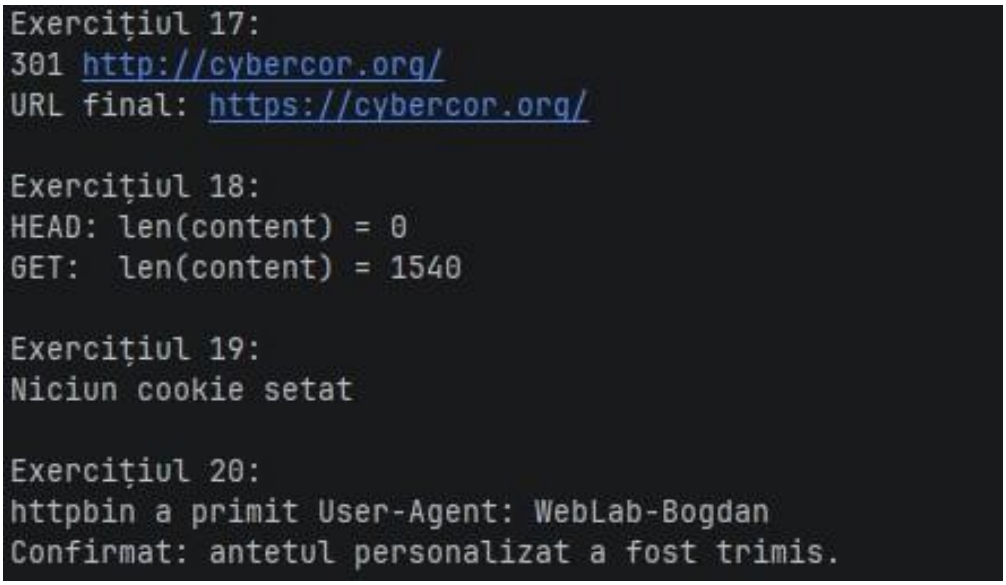
Implementare: fragmentul relevant din soluția realizată:

```
session = requests.Session()
session.headers.update({"User-Agent": "WebLab-Bogdan"})
echo = session.get(ECHO_URL + "/headers", timeout=TIMEOUT)
sent = echo.json()["headers"].get("User-Agent")
```

Explicație: Session păstrează setări comune între cererile efectuate prin acel obiect. update() modifică antetele implicite, iar /headers de pe httpbin.org returnează antetele recepționate. Verificarea se face comparând valoarea primită cu șirul setat.

Rezultat observat: httpbin.org a confirmat User-Agent: WebLab-Bogdan.

Figura 5. Output comun pentru exercițiile 17-20 din part2_methods.py.



Partea 3. Scrierea propriilor funcții (exercițiile 21-34)

Scop: Soluțiile sunt transformate în funcții reutilizabile, cu parametri, valori implicite, rezultate returnate, docstring-uri, adnotări de tip și tratarea erorilor.

Fișier: part3_functions.py

Exercițiul 21. Prima funcție

Cerință: Definirea funcției fetch(url), care trimite o cerere și returnează obiectul Response. Funcția este apelată pentru BASE_URL.

Implementare: fragmentul relevant din soluția realizată:

```
def fetch(url: str, timeout: int = 10) -> requests.Response:
    """Descarcă url și returnează obiectul răspuns."""
    return requests.get(url, timeout=timeout)

print(fetch(BASE_URL).status_code)
```

Explicație: Funcția ascunde pașii tehnici ai cererii într-un singur loc și returnează întregul obiect Response. Prin urmare, apelantul poate accesa ulterior atât codul de stare, cât și URL-ul, antetele sau corpul răspunsului.

Rezultat observat: Codul de stare afișat este 200.

Exercițiul 22. Returnarea unei singure valori

Cerință: Definirea lui get_status(url), care returnează numai codul HTTP, și apelarea funcției pentru /, /robots.txt și /sitemap.xml cu pauze de o secundă.

Implementare: fragmentul relevant din soluția realizată:

```
def get_status(url: str) -> int:
    """Returnează codul HTTP pentru url."""
    return fetch(url).status_code
```

```
for path in ["/", "/robots.txt", "/sitemap.xml"]:
    time.sleep(1)
    print(path, get_status(BASE_URL + path))
```

Explicație: Funcția preia obiectul răspuns cu fetch(), apoi extrage doar status_code. Astfel, apelantul primește o valoare simplă, de tip întreg, fără să fie nevoit să lucreze cu întregul obiect Response.

Rezultat observat: Toate cele trei adrese au returnat 200.

Exercițiul 23. Parametri impliciți

Cerință: Modificarea funcției fetch astfel încât timeout să aibă valoarea implicită 10 și testarea atât cu valoarea implicită, cât și cu timeout=3.

Implementare: fragmentul relevant din soluția realizată:

```
def fetch(url: str, timeout: int = 10) -> requests.Response:
    """Descarcă pagina cu timeout configurabil."""
    return requests.get(url, timeout=timeout)

fetch(BASE_URL)
fetch(BASE_URL, timeout=3)
```

Explicație: Parametrul timeout=10 se aplică atunci când la apel nu este transmisă o valoare pentru timeout. La al doilea apel, valoarea 3 înlocuiește temporar valoarea implicită. Acest mecanism face funcția flexibilă fără a cere repetarea aceleiași valori la fiecare apel.

Rezultat observat: Ambele cereri au returnat codul 200.

Exercițiul 24. O singură sarcină pe funcție

Cerință: Scrierea funcției get_title(html), care primește HTML și returnează textul din tagul title, fără a efectua o cerere de rețea.

Implementare: fragmentul relevant din soluția realizată:

```
def get_title(html: str) -> str:
    """Returnează titlul HTML sau un șir gol dacă lipsește."""
    start_tag = html.find("<title>")
    end = html.find("</title>")
    if start_tag == -1 or end == -1:
        return ""
    return html[start_tag + len("<title>"):end].strip()
```

Explicație: Funcția primește deja conținutul HTML și se ocupă doar de analizarea lui. Descărcarea paginii rămâne în fetch(). Separarea celor două sarcini face codul mai ușor de testat și reutilizat: get_title poate analiza și un șir HTML local, fără conexiune la internet.

Rezultat observat: Pentru pagina cybercor.org a fost extras titlul CYBERCOR.

Exercițiul 25. Docstring-uri

Cerință: Adăugarea unui docstring la get_title() și afișarea documentației prin help(get_title).

Implementare: fragmentul relevant din soluția realizată:

```
def get_title(html: str) -> str:
```

```
    """Returnează titlul paginii dintre tagurile <title> și </title>.

    Dacă pagina nu are titlu, returnează un șir gol.
    """
    ...

help(get_title)
```

Explicație: Docstring-ul este textul dintre ghilimelele triple aflat imediat după antetul funcției. El descrie scopul funcției, parametrii și comportamentul rezultatului. help() citește acest text și îl afișează ca documentație. În captura din terminal, unele diacritice apar sub forma unor secvențe Unicode, de exemplu \u0103 și \u0219; acestea sunt reprezentări ale caracterelor și nu schimbă textul docstring-ului din fișierul Python.

Rezultat observat: În captură apare documentația funcției get_title, inclusiv explicația că nu efectuează operații de rețea. Textul din captură păstrează secvențe Unicode pentru unele diacritice.

Exercițiul 26. Adnotări de tip

Cerință: Adăugarea adnotărilor de tip la funcții și apelarea get_status(123), pentru a observa dacă Python oprește automat un argument de tip greșit.

Implementare: fragmentul relevant din soluția realizată:

```
def get_status(url: str) -> int:
    """Returnează codul de stare HTTP."""
    return fetch(url).status_code

get_status(123)  # test intenționat
```

Explicație: Adnotările de tip indică tipurile așteptate, dar Python nu le verifică obligatoriu la rulare. Apelul a fost executat, iar eroarea a apărut ulterior, când requests a încercat să interpreteze valoarea 123 drept URL. Instrumente statice precum mypy sau verificările editorului pot identifica astfel de probleme înainte de execuție.

Rezultat observat: A fost afișată o eroare MissingSchema / Invalid URL pentru valoarea 123.

Exercițiul 27. Fără opriri neașteptate

Cerință: Definirea funcției page_exists(url), care returnează True sau False și prinde erorile de rețea de tip requests.RequestException.

Implementare: fragmentul relevant din soluția realizată:

```
def page_exists(url: str) -> bool:
    """Returnează True pentru un răspuns OK; False la eroare de rețea."""
    try:
        return fetch(url).ok
    except requests.RequestException:
        return False
```

Explicație: Dacă cererea reușește, funcția returnează proprietatea ok. Dacă serverul nu poate fi contactat sau URL-ul este invalid, excepția este prinsă, iar funcția returnează False în loc să întrerupă programul.

Rezultat observat: cybercor.org și ruta de test au returnat True; domeniul .invalid a returnat False. Ruta inexistentă a primit totuși un răspuns HTTP considerat OK de server.

Exercițiul 28. Returnarea unui dicționar

Cerință: Scrierea funcției check_paths(base, paths), care verifică o listă de căi și returnează un dicționar cale-cod de stare.

Implementare: fragmentul relevant din soluția realizată:

```
def check_paths(base: str, paths: list) -> dict:
    """Returnează {cale: status}; None dacă o cerere eșuează."""
    results = {}
    for index, path in enumerate(paths):
        if index > 0:
            time.sleep(1)
        try:
            results[path] = get_status(base.rstrip("/") + path)
        except requests.RequestException:
            results[path] = None
    return results
```

Explicație: Bucla parcurge fiecare cale din listă, construiește URL-ul complet și apelează get_status(). Rezultatul este păstrat într-un dicționar, în care calea este cheia, iar codul HTTP este valoarea. În cazul unei erori, se salvează None.

Rezultat observat: Pentru /, /robots.txt și /sitemap.xml a fost returnat câte 200.

Exercițiul 29. Argumente cu nume

Cerință: Definirea get_header(url, name, default="lipsește") și apelarea ei cu argumentul name transmis explicit.

Implementare: fragmentul relevant din soluția realizată:

```
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    """Returnează valoarea unui antet sau default."""
    return fetch(url).headers.get(name, default)

get_header(BASE_URL, name="Server")
```

Explicație: Argumentele cu nume arată explicit cărui parametru îi este atribuită valoarea. Dacă antetul solicitat nu există, get() returnează valoarea implicită „lipsește”.

Rezultat observat: Valoarea antetului Server afișată în captură este cloudflare.

Exercițiul 30. Antete de securitate

Cerință: Verificarea prezenței a cinci antete de securitate și returnarea unui dicționar cu valori booleene.

Implementare: fragmentul relevant din soluția realizată:

```
SECURITY_HEADERS = [
    "Strict-Transport-Security", "Content-Security-Policy",
    "X-Frame-Options", "X-Content-Type-Options", "Referrer-Policy"
]
```

```
def security_headers(url: str) -> dict:
    """Returnează True/False pentru fiecare antet verificat."""
    headers = fetch(url).headers
    return {name: name in headers for name in SECURITY_HEADERS}
```

Explicație: Lista centralizează numele antetelor care trebuie verificate. Dicționarul rezultat conține fiecare antet drept cheie și True dacă antetul există, respectiv False dacă lipsește.

Rezultat observat: În output, trei antete au valoarea True: X-Frame-Options, X-Content-Type-Options și Referrer-Policy. Celelalte două au valoarea False.

Exercițiul 31. Funcții care apelează funcții

Cerință: Scrierea score_headers(results), care numără valorile True din dicționarul exercițiului 30 și returnează un scor de forma „3/5”.

Implementare: fragmentul relevant din soluția realizată:

```
def score_headers(results: dict) -> str:
    """Returnează numărul antetelor prezente raportat la total."""
    present = sum(1 for found in results.values() if found)
    return f"{present}/{len(results)}"

score_headers(security_headers(BASE_URL))
```

Explicație: security_headers() produce dicționarul de verificare, iar score_headers() îl primește ca argument. Astfel, două funcții cu responsabilități separate sunt conectate într-un singur apel: rezultatul primei este transmis direct celei de-a doua.

Rezultat observat: Scorul afișat este 3/5.

Exercițiul 32. Două funcții, o sarcină

Cerință: Definirea funcției fetch_robots(base), care citește robots.txt, și a funcției disallowed_paths(robots_text), care extrage valorile Disallow: și acceptă și None.

Implementare: fragmentul relevant din soluția realizată:

```
def fetch_robots(base: str):
    """Returnează textul robots.txt sau None dacă nu poate fi citit."""
    try:
        response = fetch(base.rstrip("/") + "/robots.txt")
    except requests.RequestException:
        return None
    return response.text if response.status_code == 200 else None


def disallowed_paths(robots_text) -> list:
    """Extrage valorile Disallow; pentru None returnează []."""
    if not robots_text:
        return []
```

Explicație: Prima funcție se ocupă de obținerea fișierului, iar a doua doar de analizarea textului. În funcția de analiză, liniile sunt despărțite, comentariile de după # sunt eliminate, iar liniile care încep cu Disallow: sunt prelucrate. Împărțirea în două funcții separă operația de rețea de prelucrarea textului.

Rezultat observat: Captura arată mai multe valori „/” extrase din robots.txt și o listă goală pentru apelul disallowed_paths(None).

Exercițiul 33. Oricâte argumente

Cerință: Definirea response_times(*urls), care primește un număr variabil de URL-uri și returnează timpul de răspuns pentru fiecare.

Implementare: fragmentul relevant din soluția realizată:

```
def response_times(*urls: str) -> dict:
    """Returnează {url: secunde}; None când cererea eșuează."""
    times = {}
    for index, url in enumerate(urls):
        if index > 0:
            time.sleep(1)
        start = time.perf_counter()
        try:
            fetch(url)
            times[url] = round(time.perf_counter() - start, 3)
        except requests.RequestException:
            times[url] = None
    return times
```

Explicație: Parametrul *urls colectează toate argumentele poziționale într-un tuplu. Funcția parcurge adresele, măsoară fiecare cerere cu perf_counter() și stochează rezultatul într-un dicționar. Dacă o cerere eșuează, se memorează None.

Rezultat observat: Au fost măsurate două URL-uri: pagina principală și robots.txt. Timpii sunt afișați în secunde și pot varia de la o rulare la alta.

Figura 6. Output comun pentru exercițiile 21-33 din part3_functions.py.

```
Exercițiul 21:
Cod de stare: 200

Exercițiul 22:
/ -> 200
/robots.txt -> 200
/sitemap.xml -> 200

Exercițiul 23:
Cu valoarea implicită (10 s): 200
Cu timeout=3: 200

Exercițiul 24:
Titlu: CYBERCOR

Exercițiul 25:
Help on function get_title in module __main__:

get_title(html: str) -> str
    Returneaz\u0103 titlul paginii (textul dintre <title> \u0219i </title>).

    Prime\u0219te un \u0219ir HTML, nu face nicio opera\u021bie de re\u021bea. Dac\u0103 pagina nu are
    titlu, returneaz\u0103 un \u0219ir gol.


Exercițiul 26:
get_status(123) a dat eroare: MissingSchema - Invalid URL '123': No scheme supplied. Perhaps you meant https://123?

Exercițiul 27:
https://cybercor.org -> True
https://cybercor.org/this-page-does-not-exist -> True
https://this-domain-does-not-exist.invalid -> False

Exercițiul 28:
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}

Exercițiul 29:
Server: cloudflare

Exercițiul 30:
{'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}

Exercițiul 31:
Scor: 3/5

Exercițiul 32:
Căi interzise: ['/', '/', '/', '
```

Partea 4. Alte module din biblioteca standard (exercițiile 35-43)

Scop: Sunt folosite module precum urllib.parse, re, html.parser, hashlib, json, socket și ssl pentru analiza URL-urilor, HTML-ului, datelor și certificatului TLS.

Fișier: part4_modules.py

Exercițiul 35. Descompunerea unui URL

Cerință: Folosirea urllib.parse.urlparse() pentru a separa schema, domeniul, calea, query-ul și fragmentul URL-ului oferit.

Implementare: fragmentul relevant din soluția realizată:

```
parts = urlparse("https://cybercor.org/path?x=1#top")
print(parts.scheme, parts.netloc, parts.path, parts.query, parts.fragment)
```

Explicație: urlparse() returnează o structură ale cărei câmpuri pot fi citite separat. scheme este protocolul, netloc este domeniul împreună cu eventualul port, path este calea, query conține parametrii după ? și fragment identifică partea de după #.

Rezultat observat: Rezultatul: scheme=https, netloc=cybercor.org, path=/path, query=x=1 și fragment=top.

Exercițiul 36. Compunerea URL-urilor

Cerință: Transformarea legăturilor relative /about, contact.html și ../index.html în adrese complete cu urljoin().

Implementare: fragmentul relevant din soluția realizată:

```
def join_links(base: str, links: list) -> list:
    """Transformă legăturile relative în URL-uri complete."""
    return [urljoin(base, link) for link in links]
```

Explicație: urljoin() combină o adresă de bază cu fiecare legătură relativă. Funcția returnează lista rezultatelor, astfel încât fiecare adresă să poată fi folosită ca URL independent.

Rezultat observat: Au fost afișate trei adrese complete începând cu https://cybercor.org/.

Exercițiul 37. Extragerea legăturilor

Cerință: Folosirea expresiei regulate pentru a identifica valorile atributelor href din HTML și eliminarea duplicatelor.

Implementare: fragmentul relevant din soluția realizată:

```
def extract_links(html: str) -> list:
    """Extrage valorile href fără duplicate, păstrând ordinea."""
    links = re.findall(r'href="([^"]+)"', html)
    return list(dict.fromkeys(links))
```

Explicație: re.findall() caută toate aparițiile tiparului href="..." și returnează valorile dintre ghilimele. dict.fromkeys() elimină duplicatele și păstrează ordinea primei apariții. Expresia caută orice atribut href, nu doar legături din taguri <a>; de aceea poate identifica și resurse CSS din taguri <link>.

Rezultat observat: A fost găsită o singură valoare unică: /assets/index-CErP0EKt.css.

Exercițiul 38. Interne sau externe?

Cerință: Definirea split_links(links, domain), care transformă fiecare legătură în URL complet și o clasifică după domeniu.

Implementare: fragmentul relevant din soluția realizată:

```
full = urljoin(base, link)
parsed = urlparse(full)
if parsed.scheme not in ("http", "https"):
    continue
if parsed.netloc.lower() == domain.lower():
    internal.append(full)
else:
    external.append(full)
```

Explicație: Mai întâi, urljoin() transformă legăturile relative în adrese complete. Apoi urlparse() separă componentele, iar netloc este comparat cu domeniul țintă. Legăturile care nu folosesc HTTP sau HTTPS, precum mailto:, sunt ignorate.

Rezultat observat: Rezultatul afișat: 1 legătură internă și 0 externe. Legătura identificată era resursa CSS internă extrasă la exercițiul 37.

Exercițiul 39. Propriile metode

Cerință: Crearea clasei ImageFinder, care moștenește HTMLParser și suprascrie handle_starttag() pentru a colecta atributul src al tagurilor img.

Implementare: fragmentul relevant din soluția realizată:

```
class ImageFinder(HTMLParser):
    """Colectează sursele imaginilor din tagurile img."""
    def __init__(self):
        super().__init__()
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)
```

Explicație: Clasa păstrează sursele imaginilor în lista self.images. dict(attrs).get("src") extrage atributul src, iar tagurile fără acest atribut sunt ignorate. Metoda handle_starttag() este apelată automat de HTMLParser atunci când feed() citește un tag de deschidere; nu este apelată manual. Mai întâi se testează un fragment HTML cu rezultat cunoscut și abia apoi pagina reală.

Rezultat observat: Testul local a trecut și a găsit cele trei imagini așteptate. Pe HTML-ul paginii reale s-au găsit 0 taguri img; verificarea count("<img") a fost tot 0.

Exercițiul 40. Amprenta paginii

Cerință: Calcularea amprentei SHA-256 a corpului paginii și compararea rezultatelor obținute la două cereri.

Implementare: fragmentul relevant din soluția realizată:

```
def page_fingerprint(url: str) -> str:
    """Returnează hash-ul SHA-256 al conținutului paginii."""
    response = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(response.content).hexdigest()
```

Explicație: SHA-256 produce o amprentă text de lungime fixă pentru conținutul primit. Compararea celor două șiruri indică dacă octeții paginii au fost identici la cele două solicitări. Amprente diferite pot apărea dacă se schimbă pagina sau dacă serverul generează conținut dinamic.

Rezultat observat: Cele două amprente afișate au fost identice (Identice: True).

Exercițiul 41. Salvarea în JSON

Cerință: Salvarea antetelor răspunsului în headers.json cu json.dump(), apoi citirea fișierului cu json.load() și afișarea unui antet.

Implementare: fragmentul relevant din soluția realizată:

```
with open(filename, "w", encoding="utf-8") as file:
    json.dump(dict(response.headers), file, indent=2)

with open(filename, encoding="utf-8") as file:
    loaded = json.load(file)
print(loaded.get("Content-Type", "lipsește"))
```

Explicație: response.headers este convertit într-un dicționar obișnuit, care poate fi serializat în JSON. indent=2 face fișierul mai ușor de citit. json.load() reconstruiește dicționarul din fișier, iar get() permite accesarea sigură a unei chei.

Rezultat observat: Au fost salvate 18 antete, iar Content-Type a fost text/html; charset=utf-8. Fișierul headers.json este inclus în arhiva proiectului.

Exercițiul 42. Interogare DNS

Cerință: Definirea resolve(hostname), care returnează adresa IPv4 rezolvată pentru domeniu prin socket.gethostbyname().

Implementare: fragmentul relevant din soluția realizată:

```
def resolve(hostname: str) -> str:
    """Returnează adresa IPv4 pentru numele de domeniu."""
    return socket.gethostbyname(hostname)
```

Explicație: DNS transformă un nume de domeniu într-o adresă IP folosită pentru conectarea la server. Funcția delegă rezolvarea către sistemul de operare prin modulul socket.

Rezultat observat: În captura din 11.11.2026 a fost afișată adresa 104.21.91.71. Adresa poate varia în timp, deoarece domeniul poate folosi mai multe adrese IP sau infrastructură CDN.

Exercițiul 43. Expirarea certificatului

Cerință: Conectarea TLS la portul 443, citirea certificatului serverului și calcularea zilelor rămase până la expirare.

Implementare: fragmentul relevant din soluția realizată:

```
context = ssl.create_default_context()
with socket.create_connection((hostname, 443), timeout=TIMEOUT) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as secure_sock:
        cert = secure_sock.getpeercert()
expires_ts = ssl.cert_time_to_seconds(cert["notAfter"])
expires = datetime.fromtimestamp(expires_ts, tz=timezone.utc)
return (expires - datetime.now(timezone.utc)).days
```

Explicație: create_default_context() pregătește contextul TLS cu verificarea standard a certificatului. socket.create_connection() deschide conexiunea, iar wrap_socket() o transformă într-o conexiune securizată pentru domeniul indicat. Data notAfter este convertită într-un timestamp, apoi diferența față de momentul curent este exprimată în zile.

Rezultat observat: Captura arată 81 de zile rămase la momentul rulării. Valoarea este dinamică și se schimbă pe măsură ce trece timpul.

Figura 8. Output pentru exercițiile 35-43 din part4_modules.py.

```
PS D:\A SI\tp\web_lab> python part4_modules.py
Exercițiul 35:
scheme:   https
netloc:   cybercor.org
path:     /path
query:    x=1
fragment: top

Exercițiul 36:
https://cybercor.org/about
https://cybercor.org/contact.html
https://cybercor.org/index.html

Exercițiul 37:
1 legături unice
  /assets/index-CErP0EKt.css

Exercițiul 38:
Interne: 1 | Externe: 0

Exercițiul 39:
['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
0 imagini găsite
Verificare: response.text.lower().count("<img") = 0

Exercițiul 40:
Prima:   7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
A doua:  7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
Identice: True

Exercițiul 41:
Antete salvate: 18
Content-Type: text/html; charset=utf-8

Exercițiul 42:
Adresa IP pentru cybercor.org: 104.21.91.71

Exercițiul 43:
Zile rămase până la expirarea certificatului: 81
```

Partea 5. Construirea propriului modul (exercițiile 44-50)

Scop: Funcțiile reutilizabile sunt mutate în webtools.py, iar main.py importă modulul, primește un URL din linia de comandă și generează rapoarte CSV și JSON.

Fișier: webtools.py și main.py

Exercițiul 44. Crearea unui modul

Cerință: Mutarea funcțiilor fetch, get_status, get_title și security_headers în webtools.py și apelarea lor din main.py prin prefixul webtools.

Implementare: fragmentul relevant din soluția realizată:

```
import webtools

print(webtools.get_title(webtools.fetch(url).text))
```

Explicație: webtools.py devine un modul propriu, în care sunt păstrate funcțiile reutilizabile. main.py îl importă și apelează funcțiile folosind numele modulului ca prefix. Această organizare separă logica de lucru de programul principal.

Rezultat observat: Rularea programului a afișat titlul CYBERCOR.

Exercițiul 45. Importarea anumitor nume

Cerință: Schimbarea importului astfel încât main.py să importe direct get_title și security_headers, cu explicarea unui posibil conflict de nume.

Implementare: fragmentul relevant din soluția realizată:

```
from webtools import get_title, security_headers

# apel direct
title = get_title(webtools.fetch(url).text)
```

Explicație: Importul direct elimină prefixul webtools. Totuși, dacă main.py definește o funcție proprie cu același nume, ultima definiție/atribuire din fișier stabilește ce obiect va fi folosit. În comentariul din cod se explică faptul că ordinea dintre import și funcția locală poate schimba funcția asociată numelui get_title. Un alias sau folosirea prefixului modulului evită ambiguitatea.

Rezultat observat: Outputul a afișat titlul CYBERCOR și dicționarul celor cinci antete de securitate.

Exercițiul 46. Blocul main

Cerință: Adăugarea condiției if __name__ == "__main__" la finalul webtools.py pentru rularea unui autotest doar atunci când fișierul este executat direct.

Implementare: fragmentul relevant din soluția realizată:

```
if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Explicație: Când webtools.py este rulat direct, Python atribuie variabilei __name__ valoarea __main__, astfel încât autotestul este executat. Când modulul este importat de main.py, valoarea este webtools, iar blocul este sărit. Astfel, importul nu produce efecte secundare nedorite.

Rezultat observat: Comanda python webtools.py a afișat Autotest: 200.

Exercițiul 47. Constante de modul

Cerință: Definirea constantei DEFAULT_HEADERS în webtools.py, folosirea ei în funcția fetch() și importarea ei în main.py.

Implementare: fragmentul relevant din soluția realizată:

```
DEFAULT_HEADERS = {"User-Agent": "WebLab-Bogdan"}

def fetch(url: str, timeout: int = 10):
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

from webtools import DEFAULT_HEADERS
```

Explicație: Constanta păstrează antetul User-Agent într-un singur loc. Funcția fetch() o folosește pentru fiecare cerere, iar main.py o poate importa și afișa. În acest fel, aceeași configurare este reutilizată fără copierea ei în mai multe funcții.

Rezultat observat: Terminalul a afișat dicționarul cu User-Agent: WebLab-Bogdan.

Exercițiul 48. Argumente din linia de comandă

Cerință: Folosirea argparse pentru a primi URL-ul ca argument obligatoriu și transmiterea lui către funcțiile modulului.

Implementare: fragmentul relevant din soluția realizată:

```
parser = argparse.ArgumentParser(description="Raport despre un site web")
parser.add_argument("url", help="adresa site-ului")
args = parser.parse_args()
url = args.url
if not url.startswith(("http://", "https://")):
    parser.error("URL-ul trebuie să înceapă cu http:// sau https://")
```

Explicație: Argumentul url este pozițional și obligatoriu, deci trebuie furnizat la lansarea programului. parse_args() îl citește, iar verificarea prefixului acceptă doar adrese care încep cu http:// sau https://. URL-ul primit este folosit în locul unei adrese fixe BASE_URL.

Rezultat observat: Captura prezintă rularea cu python main.py https://cybercor.org, care a continuat corect. În PDF-ul cu capturi nu apare o captură separată a erorii produse când URL-ul este omis; comportamentul pentru argument lipsă este asigurat de argparse.

Exercițiul 49. Un raport CSV

Cerință: Salvarea rezultatelor check_paths() în report.csv, cu coloanele path, status și checked_at, folosind datetime.now().isoformat() pentru oră.

Implementare: fragmentul relevant din soluția realizată:

```
results = check_paths(base, paths)
with open(filename, "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerow(["path", "status", "checked_at"])
    for path, status in results.items():
        writer.writerow([path, status, datetime.now().isoformat()])
```

Explicație: csv.writer() scrie datele în format CSV. Prima linie definește numele coloanelor, iar bucla adaugă câte un rând pentru fiecare cale testată. newline="" evită linii goale suplimentare pe unele sisteme, iar encoding="utf-8" păstrează textul în codificarea potrivită.

Rezultat observat: În captura terminalului și în report.csv apar căile /, /robots.txt și /sitemap.xml, fiecare cu status 200 și cu timpul verificării.

Exercițiul 50. Proiect final: raportul site-ului

Cerință: Definirea site_report(url), care colectează codul de stare, URL-ul final, titlul, IP-ul, redirecționările, scorul antetelor de securitate, expirarea certificatului, legăturile interne/externe și căile Disallow din robots.txt; apoi afișează și salvează raportul în report.json.

Implementare: fragmentul relevant din soluția realizată:

```
report = {
    "url": url,
    "status_code": response.status_code,
    "final_url": response.url,
    "title": title,
    "ip_address": ip,
    "redirects": redirects,
    "security_score": score,
    "security_headers": headers_found,
    "certificate_days_left": days_left,
    "internal_links": len(internal),
    "external_links": len(external),
    "disallowed_paths": disallowed,
    "checked_at": datetime.now().isoformat(),
}
with open(json_path, "w", encoding="utf-8") as file:
    json.dump(report, file, indent=2, ensure_ascii=False)
```

Explicație: Funcția combină funcțiile dezvoltate anterior: fetch() pentru pagină, get_title() pentru titlu, resolve() pentru IP, redirect_chain() pentru redirecționări, security_headers() și score_headers() pentru securitate, cert_days_left() pentru certificat, extract_links() și split_links() pentru legături, respectiv fetch_robots() și disallowed_paths() pentru robots.txt. Datele sunt reunite într-un dicționar, afișate într-un format lizibil și serializate în JSON. Informațiile dependente de rețea, precum IP-ul și zilele rămase, se pot modifica între rulări.

Rezultat observat: În captura din 11.11.2026: status 200, URL final https://cybercor.org/, titlu CYBERCOR, IP 104.21.91.71, redirecționare 301 către HTTPS, scor de securitate 3/5, certificat cu 81 de zile rămase, 1 legătură internă și 0 externe. Raportul JSON afișat include și lista căilor extrase din robots.txt.

Figura 9. Rularea modulului webtools.py și a programului main.py: autotestul și outputul exercițiilor 44-50.

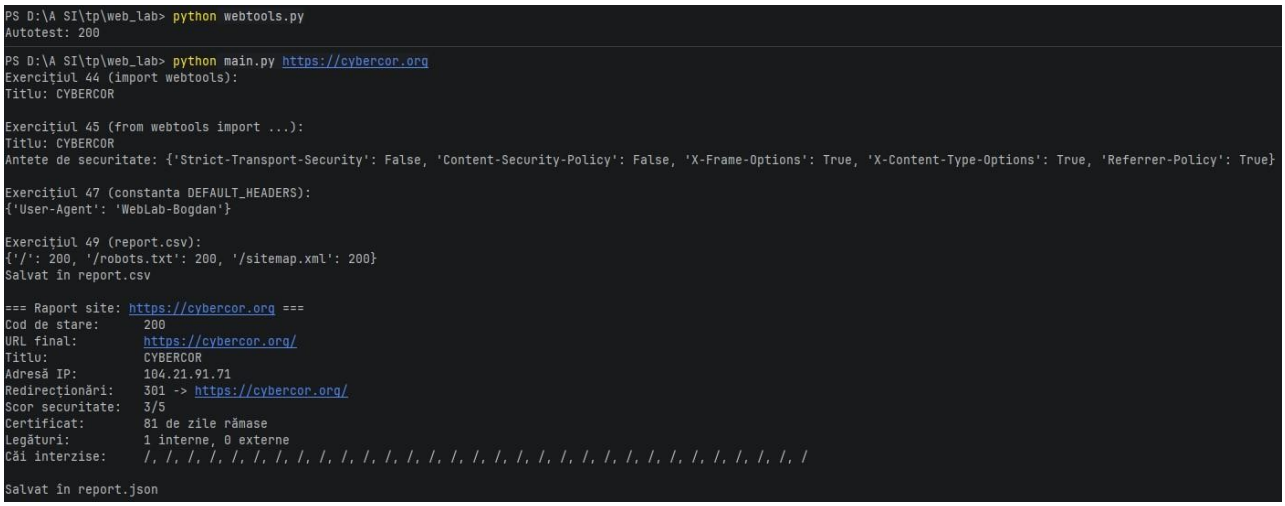
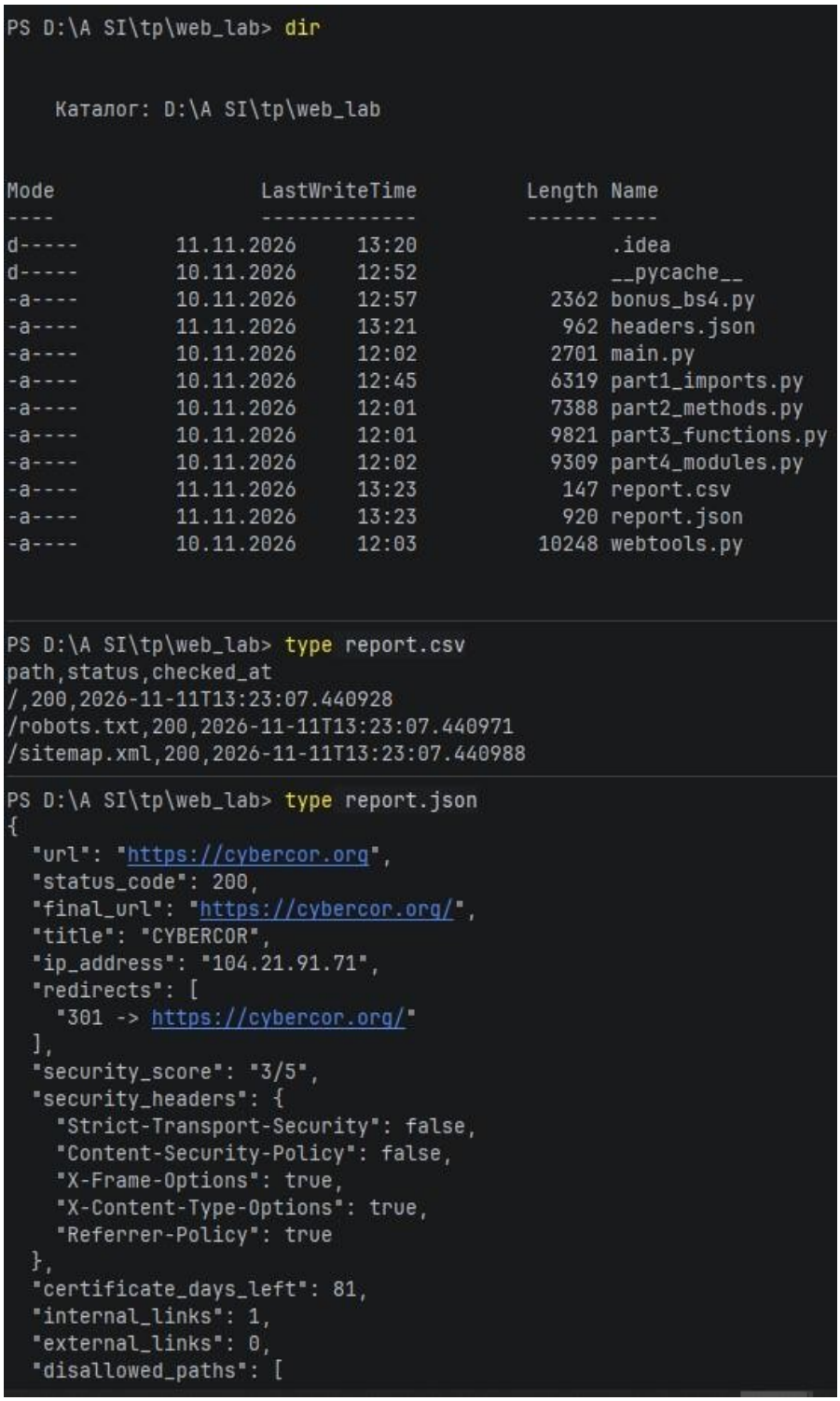

PS D:\A SI\tp\web_lab> python webtools.py
Autotest: 200
PS D:\A SI\tp\web_lab> python main.py https://cybercor.org
Exercițiul 44 (import webtools):
Titlu: CYBERCOR
Exercițiul 45 (from webtools import ...):
Titlu: CYBERCOR
Antete de securitate: {'Strict-Transport-Security': False, 'Content-Security-Policy': False, 'X-Frame-Options': True, 'X-Content-Type-Options': True, 'Referrer-Policy': True}
Exercițiul 47 (constanta DEFAULT_HEADERS):
{'User-Agent': 'WebLab-Bogdan'}
Exercițiul 49 (report.csv):
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}
Salvat în report.csv
=== Raport site: https://cybercor.org ===
Cod de stare: 200
URL final: https://cybercor.org/
Titlu: CYBERCOR
Adresă IP: 104.21.91.71
Redirecționări: 301 -> https://cybercor.org/
Scor securitate: 3/5
Certificat: 81 de zile rămase
Legături: 1 interne, 0 externe
Căi interzise: /, /, /,

Figura 10. Dovada fișierelor generate: lista directorului, conținutul report.csv și începutul report.json.



```
PS D:\A SI\tp\web_lab> dir

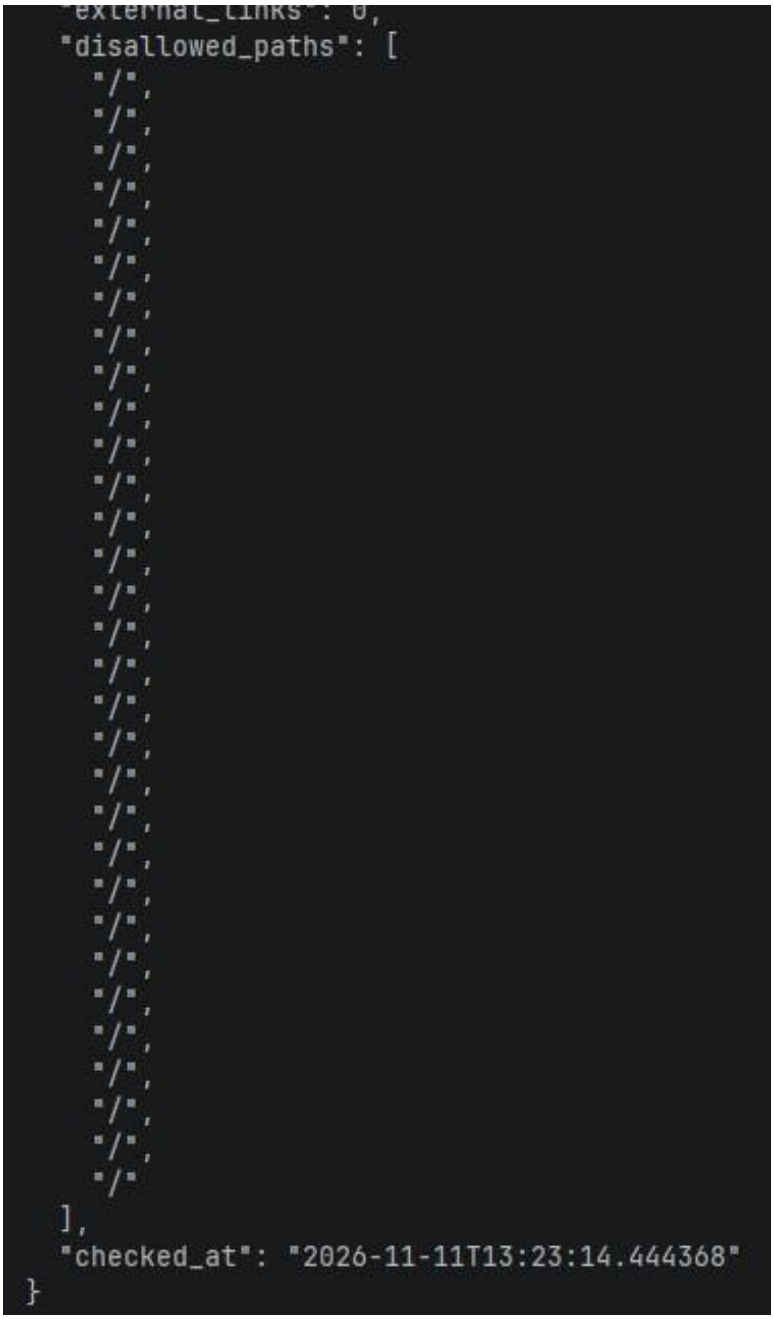

    Каталог: D:\A SI\tp\web_lab


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        11.11.2026     13:20                .idea
d-----        10.11.2026     12:52                __pycache__
-a----        10.11.2026     12:57           2362 bonus_bs4.py
-a----        11.11.2026     13:21            962 headers.json
-a----        10.11.2026     12:02           2701 main.py
-a----        10.11.2026     12:45           6319 part1_imports.py
-a----        10.11.2026     12:01           7388 part2_methods.py
-a----        10.11.2026     12:01           9821 part3_functions.py
-a----        10.11.2026     12:02           9309 part4_modules.py
-a----        11.11.2026     13:23            147 report.csv
-a----        11.11.2026     13:23            920 report.json
-a----        10.11.2026     12:03          10248 webtools.py
```

```
PS D:\A SI\tp\web_lab> type report.csv
path,status,checked_at
/,200,2026-11-11T13:23:07.440928
/robots.txt,200,2026-11-11T13:23:07.440971
/sitemap.xml,200,2026-11-11T13:23:07.440988
```

```
PS D:\A SI\tp\web_lab> type report.json
{
  "url": "https://cybercor.org",
  "status_code": 200,
  "final_url": "https://cybercor.org/",
  "title": "CYBERCOR",
  "ip_address": "104.21.91.71",
  "redirects": [
    "301 -> https://cybercor.org/"
  ],
  "security_score": "3/5",
  "security_headers": {
    "Strict-Transport-Security": false,
    "Content-Security-Policy": false,
    "X-Frame-Options": true,
    "X-Content-Type-Options": true,
    "Referrer-Policy": true
  },
  "certificate_days_left": 81,
  "internal_links": 1,
  "external_links": 0,
  "disallowed_paths": [
```

Figura 11. Continuarea aceleiași capturi: restul conținutului report.json.



Exercițiul bonus. BeautifulSoup

Cerință: Rescrierea funcțiilor de extragere a titlului și a legăturilor folosind biblioteca beautifulsoup4 și compararea cu implementarea manuală.

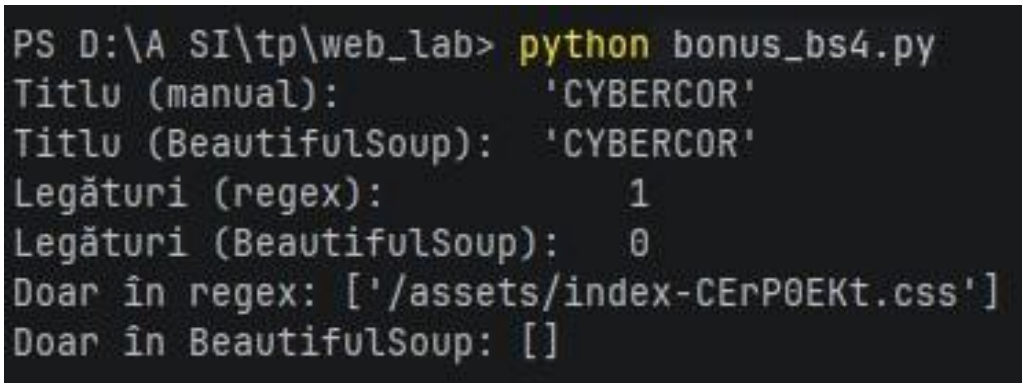
Implementare: În bonus_bs4.py sunt definite get_title_bs(html) și extract_links_bs(html). Prima folosește BeautifulSoup(html, "html.parser") și citește soup.title; a doua folosește soup.find_all("a", href=True), apoi elimină duplicatele.

```
soup = BeautifulSoup(html, "html.parser")
title = soup.title.string.strip() if soup.title and soup.title.string else ""
links = [a["href"] for a in soup.find_all("a", href=True)]
links = list(dict.fromkeys(links))
```

Explicație: BeautifulSoup interpretează structura HTML și permite alegerea elementelor după tipul tagului. Varianta manuală cu regex caută orice atribut href, în timp ce varianta BeautifulSoup selectează numai tagurile <a> care au href. Această diferență explică de ce un tag <link> pentru un fișier CSS este găsit de regex, dar nu este considerat o legătură de tip ancora HTML de BeautifulSoup.

Rezultat observat: Ambele variante au extras titlul CYBERCOR. Regex-ul a găsit 1 atribut href (resursa CSS), iar BeautifulSoup a găsit 0 taguri <a> cu href. Diferența dintre rezultate este explicată de tipurile de taguri analizate, nu de o eroare de execuție.

Figura 12. Outputul exercițiului bonus: comparația dintre varianta manuală și BeautifulSoup.



Concluzii

În urma laboratorului au fost exersate importurile de module externe și standard, citirea atributelor și apelarea metodelor obiectului Response, definirea funcțiilor reutilizabile și tratarea erorilor. În partea a patra au fost folosite module standard pentru analizarea URL-urilor și HTML-ului, calcularea amprentelor, salvarea datelor, rezolvarea DNS și verificarea certificatului TLS. În partea finală, funcțiile au fost organizate în webtools.py, iar main.py a fost configurat să primească URL-ul de la utilizator și să genereze fișierele report.csv și report.json.

Observații privind capturile și fișierele

- Capturile au fost inserate în ordinea în care apar în PDF-ul trimis. Atunci când un singur screenshot conține outputul mai multor exerciții, acesta este plasat după ultimul exercițiu din grup și este etichetat ca dovadă comună.
- Capturile pentru exercițiile 1-7, 9-16 și 21-33 conțin output grupat; capturile separate pentru exercițiile 8 și 34 sunt păstrate individual. Pentru raportul JSON, imaginile 10 și 11 sunt două continuări ale aceleiași capturi și sunt păstrate împreună.
- Valorile IP și zilele rămase până la expirarea certificatului sunt dinamice. Capturile din 11.11.2026 arată IP 104.21.91.71 și 81 de zile. În schimb, fișierul report.json din arhiva ZIP încărcată are câmpul checked_at = 2026-11-10T15:51:14.204408 și păstrează o rulare anterioară: IP 172.67.211.96 și 82 de zile. Lista de fișiere din captura de ecran indică o versiune report.json din 11.11.2026, astfel că această captură și arhiva ZIP reprezintă două stări diferite ale directorului. Valorile au fost păstrate așa cum apar în fiecare dovadă.
- În PDF nu este inclusă o captură separată a comenzii python main.py fără URL. Codul din main.py definește argumentul url ca obligatoriu prin argparse, iar captura arată rularea reușită cu URL-ul transmis explicit.